

Brite Spark 2026

Participant Handbook

Table of Contents

Brite Spark 2026.....	1
Participant Handbook	1
The short version	1
Why we run it this way	Error! Bookmark not defined.
The seven problems.....	2
Choosing your problem.....	3
A change is coming on day two	3
What every submission must include	3
How you will be judged.....	4
Using AI	5
Logistics.....	Error! Bookmark not defined.
Before you start	5

Brite Spark 2026

Participant Handbook

Welcome. Over the next two days you will take one realistic problem drawn from the kind of work Brite actually does public sector systems, benefits programs, service delivery and build something that works.

This handbook covers everything that applies to all seven problems. Your individual problem document covers the rest. Read both before you start building.

The short version

- **You work alone.** Every problem is scoped for one person over two days.
- **Pick one problem.** You are not expected to attempt more than one.
- **No paid licenses.** Anything you need should be free or open source.
- **LLM usage if needed** – use your keys and when we run the code we will use ours.
- **Any stack.** We do not care what you build it in. We care that it runs.

- **A command-line tool is a perfectly good answer.** Only Problem 4 scores the quality of the interface. For every other problem, a CLI, a script, or a notebook is a completely acceptable delivery. Do not spend day two on a front end nobody is scoring.
 - **All data is synthetic.** Nothing you are given is real. No real person's information is involved in any problem.
-

The problems

Each is a standalone problem with its own document and its own data pack. Read the summaries, then read the full document for the one you choose.

1 The Grounded Answer · *AI / RAG* A county benefits office answers the same two hundred questions every week from a policy manual that changes quarterly. Build an assistant that answers in plain language, cites the exact clause it relied on, and says “I don’t know, here is who to ask” when the manual does not cover the question.

2 Dirty Data, Real Decisions · *Data / Analytics* Case closure times have drifted and nobody knows why. The only evidence is an export from a system nobody has maintained in years. Produce a data quality assessment and answer three specific operational questions including deciding which of them the data genuinely cannot answer.

3 No Wrong Door · *Integration / Engineering* A resident’s information is scattered across systems that have never spoken to each other, and staff check each one by hand. Build a single unified view behind one API, and keep it useful when the sources misbehave.

4 The Form Nobody Can Finish · *Product / Accessibility* A long assistance application loses most of the people who start it. Rebuild the worst of it so that people can actually finish including people using a screen reader. This is the one problem where interface quality is directly assessed.

5 The Caseworker’s Morning · *Agentic AI / Guardrails* A caseworker spends the first forty minutes of every day on the same sequence of clicks. Build an agent that performs the whole sequence end to end and stops to ask a human before doing anything that cannot be undone.

6 The Overpayment Signal · *ML / Governance and Trust* A benefits program suspects some payments are improper, and investigators can only review a handful of cases a week. Produce a ranked worklist of the twenty most worth reviewing, each with a plain-language reason, and be straight about how the model behaves across the population.

7 The Reminder That Reaches · *Systems / Edge Cases* A service program loses three in ten appointment slots to no-shows. Reminders exist and plainly are not working. Build a reminder system that actually reaches people without turning into harassment.

Choosing your problem

Pick the one you find genuinely interesting. There is no easy problem in this set and no problem that scores higher than another by virtue of its track.

Two things worth knowing as you choose:

Track does not mean job. Choosing the ML problem does not stream you towards ML work, and choosing the accessibility problem is not a lesser path. The tracks exist so the set covers a spread of skills, not so you can be sorted.

Do not pick on the basis of what you already know how to build. The most common regret we expect to hear is from someone who chose the problem nearest their comfort zone, finished the obvious version by lunchtime on day one, and then had nothing interesting to do with the remaining time.

A change is coming on day two

At some point on day two, the requirements for your problem will change. Every problem gets one. You will not be told in advance what it is, and it is not the same change for every problem.

We are telling you this now, plainly, because it would be unfair not to. What we are not going to tell you is what the change is.

The point of this is not to catch you out. It is that in real delivery the requirements always move, and the interesting question about any piece of software is whether there was somewhere sensible to put the change when it arrived. Build accordingly. That does not mean building for imaginary futures over-engineering against a change you cannot predict is its own failure. It means the ordinary discipline of keeping things separable.

What every submission must include

Five things. All of them, for every problem.

1. A running demo. It must run. We will clone your repository into a clean environment and follow your README, and nothing else. If it does not start, it does not reach the judging panel this check is mechanical and happens before any human reads your code.

2. A repository with real commit history. Commit as you go.

3. DECISIONS.md. The most important file in your repository after the code. What you chose, what you rejected, and why. What you cut for time. What your solution does not do. What you would fix first. Write it as you go rather than reconstructing it at the end it reads very differently.

4. AI-USAGE.md. See the AI policy below.

How you will be judged

The rubric is published in full because we would rather you optimise for the right things than guess.

Weight	Criterion	What earns the top band
25%	Problem-solving	You handled the day-two change without rewriting everything. Your design had somewhere to put it.
25%	Judgement and honesty	DECISIONS.md names what you cut and why. You state what your solution does not do without being asked. You met the floor before reaching for anything clever.
20%	Technical capability	Real commit history across the full window. Code a stranger could pick up and extend.
20%	Quality of implementation	Runs from a clean clone on the README alone. Handles the ugly inputs, not just the happy path.
10%	Innovation	Only counts once every must-have is met. A clever idea on a broken build scores zero.

Two things follow from this table that are worth saying outright.

The floor comes first. Every problem document has a section called “The floor” listing what must be true. Until every one of those is met, nothing above them earns anything. An impressive extension built on top of an unmet requirement is worth less than a plain solution that does what was asked.

Half the weight in this rubric sits on problem-solving and honesty. Telling us that your data could not answer a question, or that your model has a problem you did not have time to fix, is not a confession it is the thing we are looking for.

Using AI

This policy is subject to final sign-off. Treat what follows as the working version.

Use whatever AI tooling you like. We are not going to pretend that is not how software gets written now, and we are not interested in a test that rewards people for hiding it.

Two conditions.

Disclose it. AI-USAGE.md should say what you used and roughly what for scaffolding, a specific function, test generation, debugging, the data cleaning, the lot. A few honest lines is enough. We are not counting.

Own it. You are responsible for every line in your repository. In the Q&A we may ask you to explain any part of your solution, why it works, and what it would do with an input you have not tried. “The model wrote that” is not an answer. If you cannot explain a piece of your own submission, it counts against you regardless of how well it works.

Before you start

A short checklist for the first hour, which is the hour most often wasted.

- Read your problem document end to end, including the “Not required” section.
- Open the data pack and actually look at it before you design anything. Every pack has surprises in it, and finding them on day two is expensive.
- Get a trivial version running end to end early, even if it does almost nothing. A pipeline that runs badly is worth more than a beautiful component that has never been connected to anything.
- Start DECISIONS.md on day one. The first entry can be why you chose your stack.

All the best !!